

Akka vs. Temporal

A comparison for teams building agentic AI

July 2026

Temporal runs your workflows; Akka runs your whole agentic system — and guarantees it. Temporal is excellent durable execution for one layer: orchestration. An agentic system also needs agents, memory, streaming, APIs, and governance — with Temporal you source, integrate, and operate each yourself. Akka delivers them as one platform, and Akka Specify can deliver and run the whole system for you as a guaranteed outcome.

99.9999%

AKKA AVAILABILITY SLA

Weeks

AKKA SPECIFY DELIVERY

Fixed

ONE ALL-IN PRICE

90%

LESS INFRASTRUCTURE

DIMENSION	TEMPORAL	AKKA
What it is	A durable-execution / workflow-orchestration engine	A full-stack agentic systems platform
Scope	Orchestration only; agents, memory, streaming, APIs, and governance are sourced and integrated by the customer	Orchestration, agents, memory, streaming, APIs, observability, and governance on one runtime
How you reach a production system	Self-integrate the full stack around orchestration, or contract a labor-led integrator	Build with spec-driven development, or Akka Specify delivers and runs it
Commercial model	Per-action metering + separate infra for every other layer + build-and-operate labor	One fixed price — platform, infrastructure, tokens, training, delivery, and operations
Outcome guarantee	Effort only; the outcome is not guaranteed	The delivered outcome is guaranteed
Availability SLA	99.9% standard; 99.99% High Availability / multi-region	99.9999% — entire platform, backed by indemnities
RTO / RPO	~20-minute RTO; sub-1-minute RPO (HA namespaces)	Sub-1-minute RTO; zero-byte RPO
AI agents	SDK integration (e.g., OpenAI Agents SDK); not native	Native agents with tools, handoffs, guardrails, interaction logging
Memory	None native — bolt-on (~200ms)	Durable in-memory, 4ms reads / sub-10ms writes
Governance / EU AI Act	Infrastructure compliance only; no AI policy enforcement, explainability, or classification	Aspect-woven runtime enforcement + full pre-production governance
Programming model	Workflow code must be strictly deterministic	Natural Java/Scala; durability handled by the runtime
Cost model	Per-action billing (\$50→\$25 per million) + separate infra for everything else	Shared compute; up to 90% lower infrastructure for the same workload
Improves after go-live	Not provided	Continuous evaluation, reinforcement learning, and distillation — up to 80% lower token cost over time

Certifications	SOC 2 Type II, GDPR, HIPAA	19 standards (SOC 2 II + public SOC 3, ISO 27001/42001, HIPAA, PCI DSS, GDPR, NIS2, DORA, EU AI Act, NIST AI RMF)
----------------	----------------------------	---

1. Temporal Is One Layer; Akka Is the Full Stack

Temporal solves durable execution of long-running workflows, and it solves it well. But an agentic AI system needs far more than orchestration, and Temporal leaves the rest to you — you select, wire, and operate the memory layer, the streaming tier, the API layer, the agent framework, and the governance stack around the orchestration core, and own every failure across those seams.

Capability	Temporal	Akka
Workflow orchestration	Yes	Yes
Native AI agents (tools, handoffs, streaming)	SDK integration only	Built in
Durable memory	None — bolt-on (~200ms)	Built in, 4ms / sub-10ms
Real-time streaming	None	Built in, backpressured, petabyte-scale
HTTP / gRPC API layer	None	Built in
Governance / policy enforcement	None	Inline, runtime-embedded
Pre-production governance	None	Classification, sign-offs, sealed posture

2. Two Ways to a Production System: Build It, or Have It Delivered

Getting from Temporal to a production agentic *system* means integrating agents, memory, streaming, APIs, and governance around the orchestration core — and then operating all of it. You do that yourself, or you contract a labor-led integrator to assemble it. Akka offers a second path entirely: **Akka Specify** takes your specifications and delivers and operates the whole governed system for you as a guaranteed outcome — in weeks, for one fixed price.

	With Temporal	With Akka Specify
Model	Self-integrate the full stack around orchestration, or a labor-led integration engagement	You provide the specifications
Who does the work	Your engineers, or forward-deployed / staff-augmentation contractors	Akka generates, governs, delivers, and runs the system
Timeline	3–12 months to a full system, by construction	Weeks, not quarters
Billing	Per-action metering + separate infra + build-and-operate labor	One fixed price
Afterward	You own and operate every layer and every seam	Kept up, safe, and improving — operated by Akka
Guarantee	Effort is guaranteed; the outcome is not	The delivered outcome is guaranteed

Temporal gives you a best-in-class orchestration component to build around. Akka Specify hands you the finished, governed, running system — and keeps it available, safe, and improving under one agreement.

3. Availability and Disaster Recovery

Temporal Cloud has materially strengthened its HA/DR: High Availability and multi-region namespaces publish a [99.99% availability SLA](#) (99.9% standard) with sub-1-minute RPO and ~20-minute RTO via automatic failover. That is real. Two gaps remain, and both matter for mission-critical agentic systems.

Metric	Temporal	Akka
Availability SLA	99.99% (HA / multi-region)	99.9999%
Allowed downtime / year	~52 minutes	~31 seconds
RTO	~20 minutes (auto failover)	Sub-1 minute
RPO	Sub-1 minute	Zero byte
SLA scope	The orchestration layer	The entire platform
Who operates it	The customer (the assembled stack)	Akka SREs, 24/7

Temporal's SLA covers the orchestration layer. The memory, streaming, API, and governance services a customer bolts on operate under their own SLAs — or none. When Akka delivers through Akka Specify, that entire running system is operated by Akka under one SLA.

4. The Determinism Tax

Temporal requires all workflow code to be **strictly deterministic** — no direct I/O, no unguarded randomness, no non-deterministic library calls — because recovery replays history. That constraint is at odds with agentic AI:

- LLM calls are non-deterministic by nature and must be carefully wrapped in activities.
- Any library with internal randomness must be audited or replaced.
- A non-determinism bug surfaces as a replay failure in production; debugging it needs deep Temporal expertise.

Akka provides durability and fault tolerance without imposing a determinism constraint on application code: state is event-sourced and the runtime handles recovery, so teams write natural business logic.

6. Governance and the EU AI Act

Temporal holds SOC 2 Type II, GDPR, and HIPAA — infrastructure-security compliance. It publishes no AI-governance capability: no real-time policy enforcement, no decision explainability, no human pause/override of a running process, no immutable interaction ledger, no pre-deployment classification, no sealed audit artifact.

The penalties are enforceable now

Violation	Maximum Fine
Prohibited AI practices (Art. 5)	EUR 35M or 7% global turnover
High-risk obligations (Art. 9-15)	EUR 15M or 3% global turnover

High-risk AI carries a 10-year logging-retention obligation (Art. 72).

5. Cheaper to Operate — and It Keeps Getting Cheaper

AI systems built with Akka are up to **90% cheaper to operate** than the equivalent assembled stack. Temporal bills **per action** — every step, signal, query, and timer is metered at \$50 per million (stepping to \$25 at high volume) — and that meter covers only orchestration; memory, streaming, APIs, observability, and governance are billed separately.

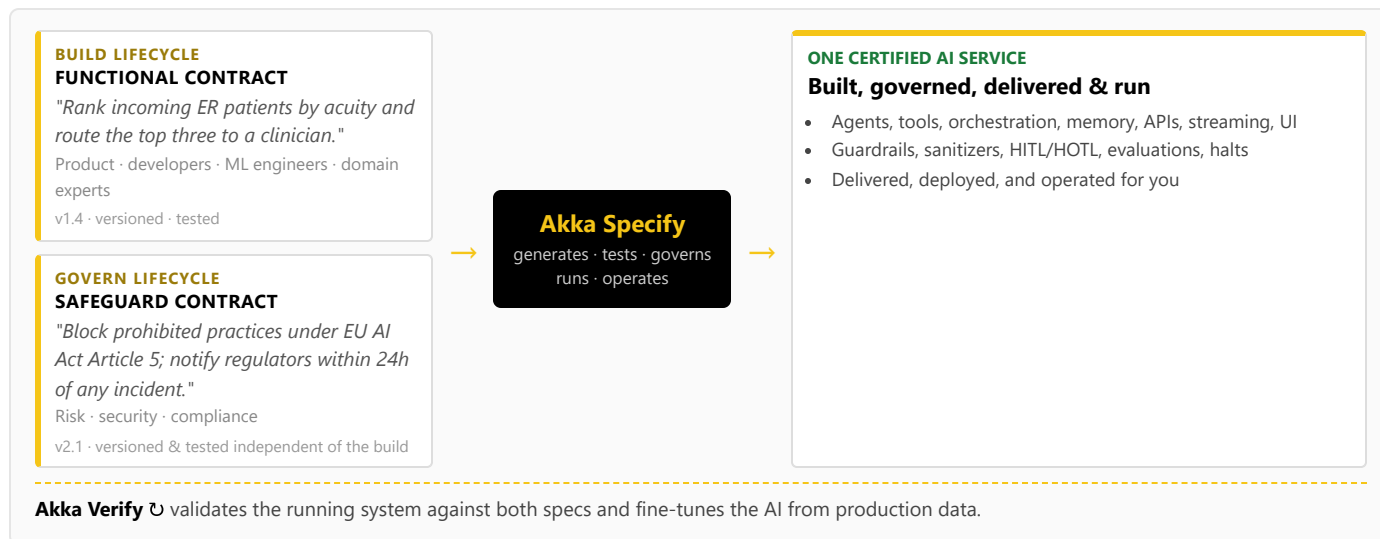
Akka runs all of it on one shared-compute runtime (actor concurrency ~10T tokens/core/year vs ~2T; ~80% less compute than Python; Manulife: up to 300% more concurrency, 30–50% faster after porting from Python). The delivered outcome also **compounds**: Akka Verify's continuous evaluation, reinforcement learning, and distillation cut token cost up to 80% over time. The spend is predictable — **one fixed price**, not per-action metering that moves with load plus separate build-and-operate labor.

How Akka governs

At the runtime: inline guardrails, policies, LLMs-as-a-judge, and sanitizers; hash-chained immutable evidence; HITL/HOTL control; classification against 186 regulations and 877 controls before a system ships; multi-persona sign-offs; a sealed Governance Posture Package; and Akka Verify proving conformance from the running system. Governance Temporal would have to bolt on, Akka enforces inline — and delivers it already wired when the system is built through Akka Specify.

7. One Certified System — Built, Governed, Delivered, and Run

Building on Temporal means engineers writing deterministic workflow code; there is no path for a product manager, domain expert, or risk officer to contribute, and no built-in governance lifecycle. Akka runs two independent lifecycles on one platform via **Akka Specify**:



Akka generates, tests, governs, **runs, and operates** one certified service from both specs, which are versioned and tested independently by different audiences. Through Akka Specify, that certified system is delivered and operated as a guaranteed outcome — a delivery model and an independent governance lifecycle Temporal has no equivalent for.

8. Real-Time Streaming at Petabyte Scale

Temporal has no streaming engine; real-time pipelines are provisioned separately. Akka's streaming is built into the runtime — continuous, backpressured, **petabyte-scale, in-memory**, with no external broker — powering both agent feedback loops and high-throughput data processing (the engine behind Tubi's real-time hyper-personalization at 5 billion tokens per second).

9. For the Buyer: Risk, Compliance, and Accountability

Buyer concern	Temporal	Akka
Certifications & audits	SOC 2 Type II, GDPR, HIPAA	19 standards — SOC 2 II + public SOC 3, ISO 27001/42001, HIPAA, PCI DSS, GDPR, NIS2, DORA, EU AI Act, NIST AI RMF — plus annual pen tests, SBOMs, 40+ policies (trust.akka.io)
Delivery & outcome	Self-integrate the full stack, or a time-and-materials integrator; effort is guaranteed, the outcome is not	Akka Specify delivers and operates the system for one fixed price; the outcome is guaranteed
Scope of accountability	The orchestration layer; you integrate and operate the rest	One platform, one SLA, 24/7 SRE — Akka owns the running system
Risk transfer	Standard cloud terms	Availability and data-integrity guarantees backed by contractual indemnities
Track record & funding model	Venture-funded: \$300M Series D at a \$5B valuation (2026); scaling on investor capital toward profitability	Profitable and self-funding; 18 years and 100,000+ deployments (52 banks); Dell Technologies Capital is largest shareholder, a customer, and an AI partner
Budget predictability	Per-action metering that scales with load	One fixed price finance can forecast

Both are well-capitalized — but Akka funds its roadmap and support from profit, not the next round, so the platform you standardize on does not depend on a venture timeline. The decision is scope and accountability: Temporal gives you a best-in-class orchestration layer to build a platform around; Akka gives you the platform — and, through Akka Specify, will deliver and run the whole system for you.

Customers Running Agentic and Real-Time Systems on Akka

Manulife

2,000

developers across 100 projects on one governed platform

Tubi

5B tok/s

real-time hyper-personalization engine

Swiggy

71ms

order-assignment AI, ~50% latency reduction

John Deere

1,000+

tractor sensors turned into real-time insight

Verizon

750%

order-processing capacity gain; 6s → 2.4s response

Common Questions

We don't have a team to build the full agentic stack around Temporal — what are our options?

Two. Build it on the platform with spec-driven development, or have Akka Specify deliver and operate it for you. You provide plain-language specifications; Akka generates, governs, delivers, and runs the whole system — agents, memory, streaming, APIs, orchestration, and governance — as a guaranteed outcome, for one fixed price. With Temporal you self-integrate that stack or contract a labor-led integrator you then own.

How is Akka Specify different from hiring an integrator to build our agentic system on Temporal?

An integration engagement sells effort — time-and-materials over months — and hands back a system you own and operate; the outcome is not guaranteed. Akka Specify sells the outcome: a governed system delivered in weeks for one fixed price, then kept up, safe, and improving by Akka. You own the specifications, not the operational burden.

We already run Temporal in production. Why add Akka?

Temporal is strong for durable workflow orchestration. If you are moving into agentic AI you also need native agents, durable memory, streaming, and runtime governance — which Temporal does not

Temporal raised \$300M at a \$5B valuation and targets agentic AI. Doesn't that close the gap?

Strong funding reflects real demand for durable execution. But Temporal's agentic story is an SDK integration on top of orchestration; Akka is built for agentic AI end to end — native agents, built-in memory, embedded governance, and a full-stack runtime. Capitalization does not add the missing layers.

Can we add governance on top of Temporal?

You can add log-analysis tools, but the EU AI Act expects enforcement inline to the runtime: immutable records witnessed as they happen, human override on running processes, and authorization capture at execution time. Bolt-on tools read logs after the fact and cannot gate a deployment or classify a system before it ships. Akka embeds all of this and covers pre-deployment governance.

Is Temporal cheaper because it is open source?

Temporal's core is open source, but production Temporal means Temporal Cloud (per-action billing) or self-hosting (you build and operate HA/DR, observability, and everything else). Akka's shared-compute model is up to 90% cheaper to operate than the equivalent assembled stack, on one fixed price.

provide. Akka can run alongside existing Temporal workflows while you build the agentic layer on one platform.

Sources

Temporal Cloud SLA: docs.temporal.io/cloud/sla — 99.9% standard; 99.99% HA / multi-region

Temporal HA/DR: docs.temporal.io/cloud/high-availability, /rpo-rto — sub-1-min RPO, ~20-min RTO, automatic failover

Temporal Cloud pricing: docs.temporal.io/cloud/pricing — \$50→\$25 per million actions (\$0.00005/action)

Temporal Series D: \$300M at \$5B valuation, Feb 2026 (businesswire / geekwire); 380% YoY revenue growth

Temporal security: temporal.io/security — SOC 2 Type II, GDPR, HIPAA

Akka Specify (spec-driven delivery): akka.io / akka.io/llms.txt — you provide the specifications; Akka generates, tests, governs, delivers, and operates the system as a guaranteed outcome; one fixed price covering platform,

infrastructure, tokens, training, delivery, and operations; delivered in weeks; continuous improvement via reinforcement learning and distillation (up to 80% lower token cost, higher accuracy)

Akka trust center: trust.akka.io — 19 compliance standards; SOC 2 II + public SOC 3; annual pen tests, SBOMs, 40+ policies

Akka performance: akka.io/blog/go-slow-to-go-fast — Manulife up to 300% more concurrency, 30–50% faster; ~10T vs ~2T tokens/core; ~80% less compute than Python

Akka platform: 99.9999% availability, active-active HA/DR, sub-1 min RTO, zero byte RPO (contractual indemnities); 186 regulations / 877 controls; 100,000+ deployments / 18 years; profitable; Dell Technologies Capital